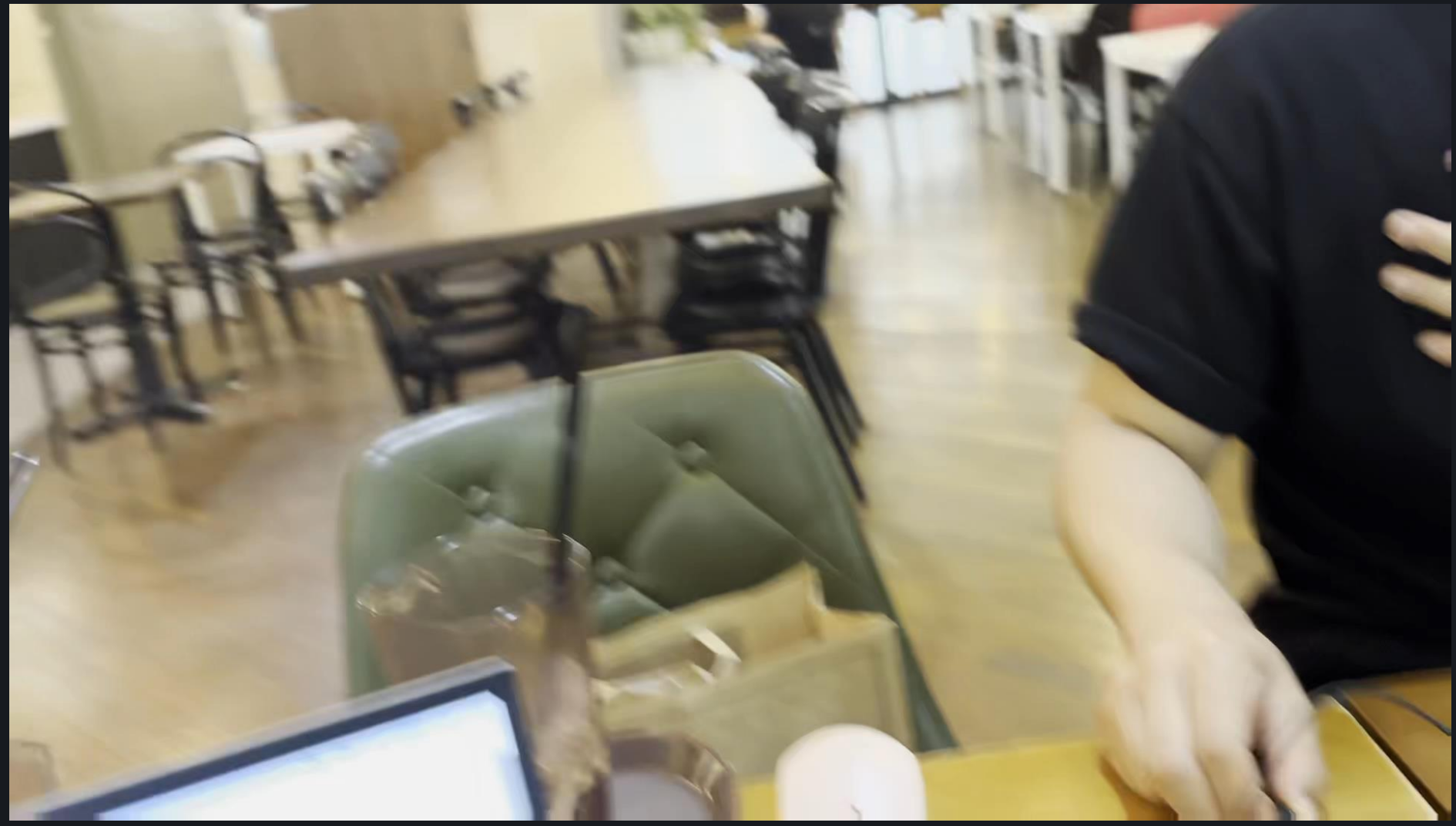
HUFS IoT TERM PROJECT

Edge AI
Threat-Sound Detection

Raspberry Pi 4 / YAMNet / Frame-aligned Fusion

Taehyun Jang, Yuna Ko

Demo Video



Why listen for threats?

01 COVERAGE GAP

Cameras and motion sensors have blind spots. Sound provides an independent signal outside the field of view.

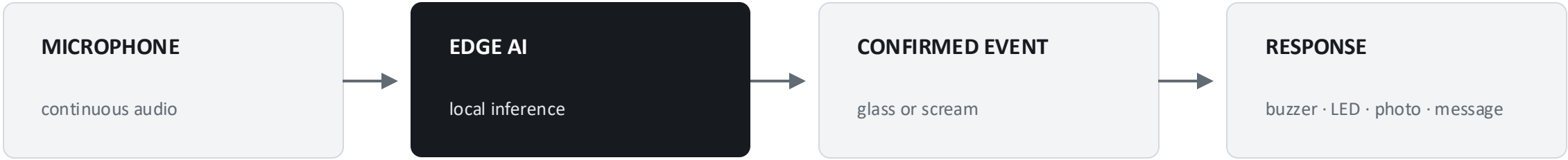
02 EDGE PRIVACY

Raw audio is analyzed locally on Raspberry Pi. Only detected events and evidence need to leave the device.

03 AUTOMATED RESPONSE

A confirmed event can trigger GPIO alerts, camera capture, logging, Telegram, and email notifications.

DETECTION-TO-ACTION FLOW



Project idea and objectives

MAIN CONCEPT

Use sound as an additional security sensor, classify threats locally, and connect detection directly to IoT response.

01 DETECT

Recognize breaking glass and screams while rejecting ordinary environmental audio.

02 PROTECT PRIVACY

Perform inference on the Raspberry Pi instead of continuously uploading raw audio.

03 RESPOND

Translate a confirmed sound event into physical alerts, evidence capture, and remote notification.

SUCCESS CRITERION Balance both threat recalls against normal false alarms while remaining practical on Raspberry Pi.

End-to-end system design

A rolling audio window is converted into two complementary signals, fused into one event decision, and connected to IoT actions.

SENSING LAYER

microphone and rolling buffer

AI LAYER

YAMNet + custom classifier

POLICY LAYER

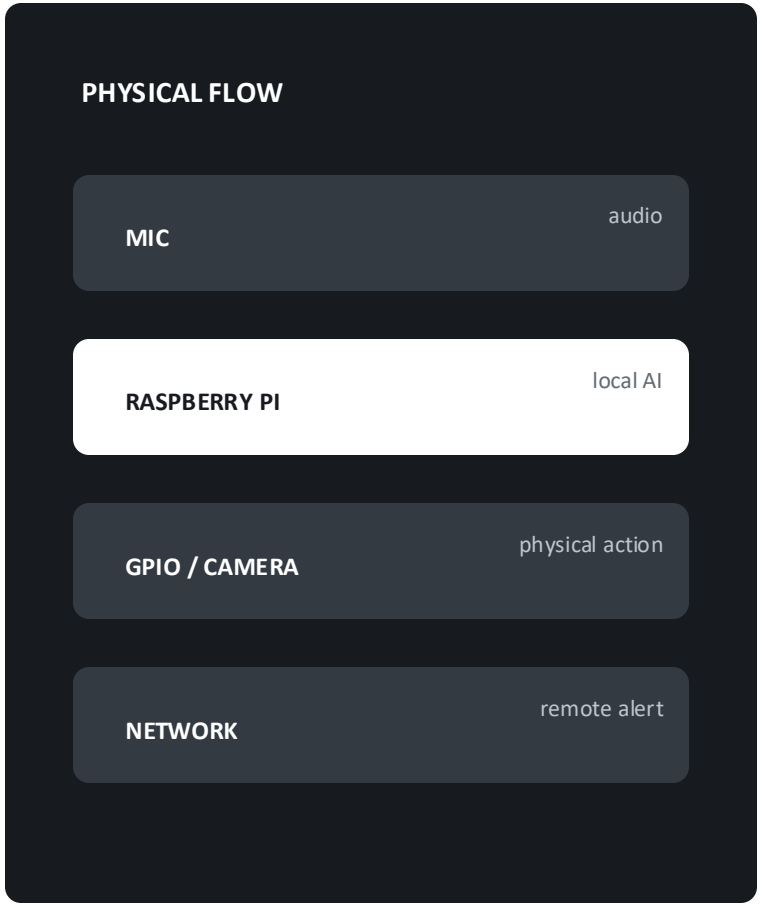
thresholds and temporal Fusion

IOT LAYER

local and remote alerts

Hardware architecture

Component	Role	Implementation detail
Raspberry Pi 4	Edge computer	Runs YAMNet, H5 classifier, Fusion, and alert control
USB lavalier microphone	Audio sensing	Deployment-domain recordings and live 16 kHz input
Pi Camera	Evidence capture	Takes a photo after a confirmed event
LED + buzzer	Local warning	Controlled through Raspberry Pi GPIO
Network connection	Remote notification	Telegram and email transmission
PC + PuTTY	Remote terminal	Starts and monitors the Pi process



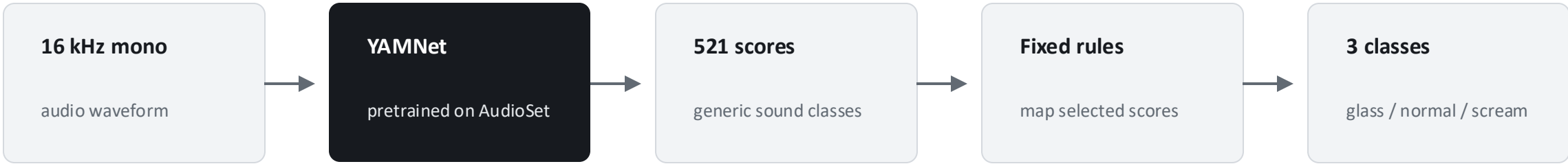
Software architecture and implementation

Code / component	Responsibility	Implementation
realtime_detect.py	Live inference loop	Captures audio blocks, maintains the rolling window, and invokes the final decision
audio_pipeline.py	Audio preprocessing	Resampling, mono conversion, fixed-length loading, and stream constants
TF Hub YAMNet	Feature extraction	Returns 1,024-D embeddings and 521 AudioSet scores in one pass
candidate_c_20260619.h5	Task classifier	Dense 512 → 256 → 128 → 3 with regularization
detection_policy.py	Final Fusion policy	Strong sustained path plus same-frame classifier/YAMNet agreement
alert modules	IoT response	gpio_alert.py, camera_alert.py, telegram_alert.py, and email_alert.py

IMPLEMENTATION STACK · Python · TensorFlow / Keras · TensorFlow Hub · NumPy · SoundDevice · Raspberry Pi GPIO

Baseline: YAMNet-only control

The custom head is removed. Generic AudioSet scores are mapped directly to our three classes using fixed rules.



CONTROL RESULT

Macro F1

0.756

Scream recall

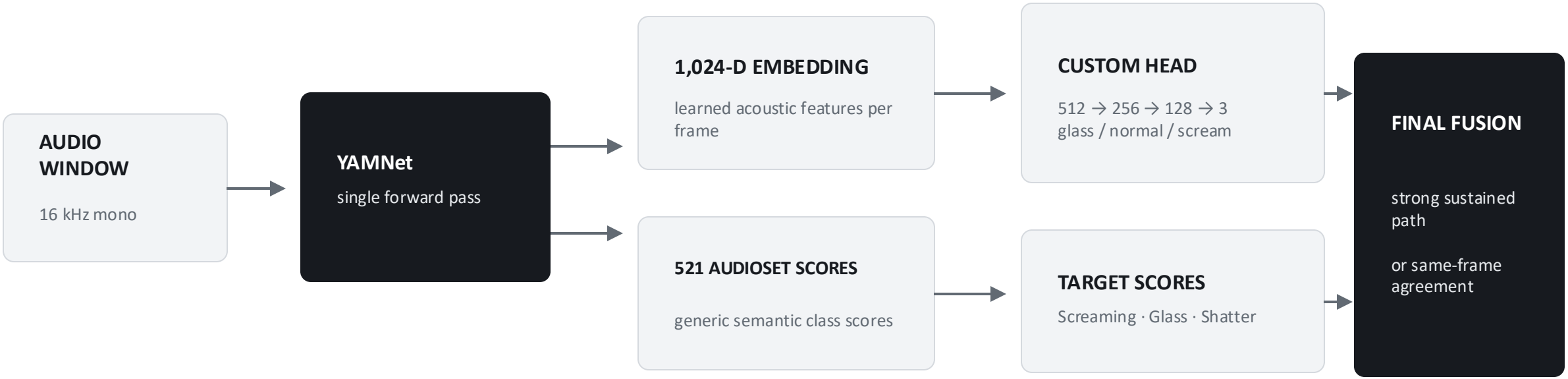
50.0%

LIMITATION

Generic semantic scores are useful, but they are not sufficiently specialized for our target classes and deployment conditions.

Final architecture: one YAMNet pass, two outputs

The neural network and the decision policy are separate: the custom head learns the task; Fusion decides when to trigger.



KEY IDEA The 1,024 values are features and the 521 values are generic class scores. Both come from the same YAMNet execution

Frame-aligned Fusion decision policy

The custom classifier provides the primary task-specific signal. YAMNet confirms only the more sensitive path.

PATH A · STRONG

High classifier confidence
+ consecutive-frame requirement
→ can trigger directly

OR

PATH B · ALIGNED

Sensitive classifier condition
+ relevant YAMNet score
+ same frame
→ trigger only when both agree

→

FINAL EVENT

glass / scream
or normal

Class	Strong custom	Strong frames	Aligned custom	YAMNet score	Aligned frames
Glass	0.95	2	0.40	0.10	1
Scream	0.55	3	0.85	0.02	1

Data sources and class definition

AUDIOSET

Google AudioSet supplies labeled 10-second YouTube segments with broad acoustic diversity.

Screaming · Glass · Shatter

DEVICE RECORDINGS

Direct Raspberry Pi microphone samples capture the actual microphone and room domain.

reviewed failures · live examples

NORMAL HARD NEGATIVES

Ordinary sounds are deliberately included to measure and reduce false alarms.

speech · crowds · music · impacts · footsteps

STANDARDIZATION

16 kHz mono · 3-second clips · strict target labels · original-source identity retained

Dataset, augmentation, and split

DATA SOURCES

AudioSet YouTube segments
+ Raspberry Pi microphone recordings
+ realistic normal hard negatives

TRAINING AUGMENTATION

GAIN volume variation

NOISE additive background noise

FILTERING bandwidth variation

MIXING normal backgrounds · varied SNR · random position

LEAKAGE CONTROL

Counts use original-source groups. Duplicates and augmented versions stay with their source.

Class	Train sources	Augmented audio	Final train audio	Val sources	Test sources
Glass	113	1,695	1,808	25	29
Normal	433	1,299	1,732	93	98
Scream	186	1,464	1,650	38	38
Total	**732**	**4,458**	**5,190**	**156**	**165**

Training and evaluation methodology

1	**SOURCE SPLIT**	Split original-source groups before evaluation. Related or augmented audio cannot cross splits.
2	**FEATURE EXTRACTION**	Freeze YAMNet and cache frame-level 1,024-D embeddings.
3	**CUSTOM TRAINING**	Train the dense head with class weighting, dropout, batch normalization, L2, and early stopping.
4	**POLICY SELECTION**	Compare three fixed Fusion candidates on validation; require both event recalls to reach 70%.
5	**FINAL EVALUATION**	Apply the selected model and policy to the strict-label test groups.

Trial and error changed the system

Stage	What failed	What we learned	Final change
MIDTERM	Clip-level mean pooling	Short glass impacts were diluted by surrounding normal frames.	Retain frame-level evidence.
DATA AUDIT	Broad breaking-sound labels	Generic labels introduced noisy glass examples.	Keep verified glass and shatter sources; recycle reviewed failures.
GATING CHECK	Relaxed threshold without YAMNet	Sensitivity increased, but many normal sounds triggered.	Require same-frame YAMNet agreement on the relaxed path.

ENGINEERING LESSON · Better evaluation discipline and deployment-domain feedback mattered as much as model complexity.

Baseline vs Final Fusion

Same strict-label test split · 165 independent source groups

System	Macro F1	Glass recall	Scream recall	Normal FAR
YAMNet-only	0.756	69.0%	50.0%	5.1%
Final Fusion	**0.837**	**75.9%**	**78.9%**	**9.2%**

+28.9 pts

scream recall

+0.081

Macro F1

TRADE-OFF

normal FAR increased by 4.1 pts

Results and project contributions

01 TASK SPECIALIZATION

A compact custom head converted general YAMNet embeddings into a detector specialized for glass, normal, and scream.

02 FRAME-LEVEL FUSION

Same-frame agreement preserved short transient evidence while filtering weak relaxed-threshold detections.

03 EDGE DEPLOYMENT

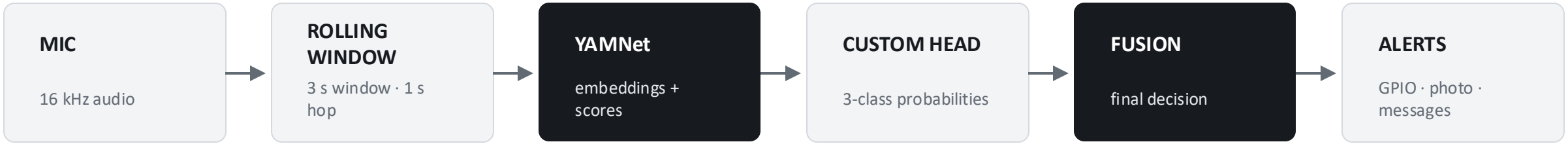
The complete model-processing path ran on Raspberry Pi with a measured mean of 287 ms.

04 ENGINEERING DISCIPLINE

Source-separated splits, strict labels, and a microphone-domain feedback loop made the result more defensible.

BENEFIT · Local inference reduces continuous audio transmission and supports immediate physical and remote response.

Real-time Fusion on Raspberry Pi



MEASURED MODEL-PROCESSING SCOPE

WAV load → YAMNet → H5 classifier → Fusion decision

Measured over 30 files through PuTTY SSH on Raspberry Pi.

287 ms

mean model-processing time

293 ms p95

Where we can improve next

01 DATA

More real microphone screams
Impact, speech, and footstep negatives

Live failure-capture loop
Site-specific validation

02 MODEL

Selective YAMNet fine-tuning
Class-specific calibration

Compact temporal model
Uncertainty rejection

03 DEPLOYMENT

Fresh external validation
Sustained-load Raspberry Pi test

False alarms per hour
End-to-end alert latency

Security that listens.

01 Transfer learning made edge deployment practical.

02 The custom head specialized YAMNet embeddings for three classes.

03 **Frame-aligned Fusion combined task-specific and semantic evidence.**

04 Fresh external validation is the next required step.